

```

/**
 * FILENAME: CSC 340-03 12:30 section, random number generation & application
 * CONTRIBUTORS: Phong Le, Michael Leong, Rathang Pandit
 * DATE: Last Updated 10/2
 * DESCRIPTION: 3 part problem; Generate a histogram of random numbers with a
standard distribuion, Generate a histogram of random numbers with a uniform
 *                distribution, Assign a list of students to groups
 */

```

```

#include <iostream>
#include <vector>
#include <random>
#include <algorithm>

```

```

/* Helper Function: roundToBin(num) */
double roundToBin(std::vector<std::pair<double, int>> histo, double num){
    //declarations
    double prevBin = histo.at(0).first; //bin at left
    double nextBin;
    //iterate through bins
    for(int i = 1; i < histo.size(); i++){
        nextBin = histo.at(i).first; //bin at right
        //num is between two bins
        if(num < nextBin){
            //check which of the two bins
            if(nextBin - num <= num - prevBin){ //close to nextBin
                return nextBin;
            }
            else{ //closer to prevBin
                return prevBin;
            }
        }
        prevBin = nextBin;
    }
    //finished loop but the number was bigger than the last bin
    return histo.at(histo.size()-1).first;
}

```

```

/** FIRST PROBLEM
 * desc: Generate a histogram detailing the frequencies of the random numbers that
fall within a certain range
 *       Bins denoted with 1) center value, 2) frequency. bins are equidistant
(mean +/- x*stdDev) and equal in width (range)
 * notes: The canvas instructions mention rounding into the nearest bin, however
she said that if the widths of the bins dont
 *       touch the other bins, a number generated in between the bins will be
discarded. so presumably that means if the value
 *       is not exactly the center but is within the width, it is counted towards

```

the bin of the centered value.

```
* in: mean
*     stdDev
*     numBins
*     numSamples
*     binWidth (no bigger than stdDev)
* defaults: numSamples : 20000
*           binWidth : 1 (catch all numbers)
* out: a collection of elements, each containing a designation (binNum) and a
frequency
*/
```

```
std::vector<std::pair<double, int>> normalDistribution(double mean, double stdDev,
int numBins, int numSamples, double binWidth){
```

```
    int i;
    int j;
```

```
    //check for defaults
```

```
    if (numBins < 9){
        numBins = 9;
```

```
    }
```

```
    if (numSamples == -1){
        numSamples = 20000;
```

```
    }
```

```
    if (binWidth == -1 || binWidth > stdDev){
        binWidth = stdDev;
```

```
    }
```

```
    /* Generate numbers into a histogram that follow normal distribution (using
mean and stdDev) */
```

```
    //declare histogram as a vector of pairs(bin center, frequency) of size numBin
std::vector<std::pair<double, int>> histo;
```

```
    histo.resize(numBins);
```

```
    //initialize pairs in histogram
```

```
    //start from center (mean) to end
```

```
    j = 0;
```

```
    for(i = numBins/2; i < numBins; i++){
        histo.at(i).first = mean+j*stdDev;
        histo.at(i).second = 0;
        j++;
```

```
    }
```

```
    //start from left of center to beginning
```

```
    j = -1;
```

```
    for(i = numBins/2 -1; i >= 0; i--){
        histo.at(i).first = mean+j*stdDev;
        histo.at(i).second = 0;
        j--;
```

```
    }
```

```
    //declare random number engine
```

```

std::random_device rando{};
std::mt19937 engine{rando()};
//declare distribution
std::normal_distribution<double> distr{mean, stdDev};
//declare a temp variables before loop
//to store random number
//the nearest bin to random number
double randNum;
double nearBin;

//loop numSample times
for(i = 0; i < numSamples; i++){
    //generate a random number
    randNum = distr(engine);

    //if number is within bin width of the nearest bin
    //nearest bin center
    nearBin = roundToBin(histo, randNum);

    //if number is within the upperbound of the lower bin
    if(randNum <= nearBin + binWidth/2){
        //find the bin in the histo
        for(j = 0; j < numBins; j++){
            if(histo.at(j).first == nearBin){
                //add to the bin frequency
                histo.at(j).second++;
            }
        }
    }

    //if number is within the lowerbound of the upper bin
    else if(randNum >= nearBin - binWidth/2){
        //find the bin in the histo
        for(j = 0; j < numBins; j++){
            if(histo.at(j).first == nearBin){
                //add to the bin frequency
                histo.at(j).second++;
            }
        }
    }

    //otherwise, ignore the random number because it does not land within the
    widths of the nearest bin
}

return histo;
}

```

/** SECOND PROBLEM

```

* desc: create and return a histogram of random numbers in a uniform distribution
given a range, number of bins, number of samples, and catching range of bins
* note: distance between bins are (upper-lower)/numBins
* in: lower
*       upper
*       numBins (no less than 21)
*       numSamples
*       binWidth (max is the distance between bins)
* defaults: numBins : 21
*           numSamples : 20000
*           binWidth : (upper-lower)/numBins
* out: a collection of elements, each containing a designation (binNum) and a
frequency
*/
std::vector<std::pair<double, int>> uniformDistribution(double lower, double upper,
int numBins, int numSamples, double binWidth){
    int i;
    int j;

    //check for defaults
    if (numBins < 21){
        numBins = 21;
    }
    if (numSamples == -1){
        numSamples = 20000;
    }
    if (binWidth == -1 || binWidth > (upper-lower)/numBins){
        binWidth = (upper-lower)/numBins;
    }

    //declare histogram as a vector of pairs(bin center, frequency) of size numBin
    std::vector<std::pair<double, int>> histo;
    histo.resize(numBins);
    //initialize pairs in histogram
    for(i = 0; i < numBins; i++){
        histo.at(i).first = i * ((upper-lower)/numBins);
        histo.at(i).second = 0;
    }

    //declare random number engine
    std::random_device rando{};
    std::mt19937 engine{rando()};
    //declare distribution
    std::uniform_real_distribution<double> distr{lower, upper};
    //declare a temp variables before loop
    //to store random number
    //the nearest bin to random number
    double randNum;
    double nearBin;

```

```

//loop numSample times
for(i = 0; i < numSamples; i++){
    //generate a random number
    randNum = distr(engine);

    //if number is within bin width of the nearest bin
    //nearest bin center
    nearBin = roundToBin(histo, randNum);

    //if number is within the upperbound of the lower bin
    if(randNum <= nearBin + binWidth/2){
        //find the bin in the histo
        for(j = 0; j < numBins; j++){
            if(histo.at(j).first == nearBin){
                //add to the bin frequency
                histo.at(j).second++;
            }
        }
    }

    //if number is within the lowerbound of the upper bin
    else if(randNum >= nearBin - binWidth/2){
        //find the bin in the histo
        for(j = 0; j < numBins; j++){
            if(histo.at(j).first == nearBin){
                //add to the bin frequency
                histo.at(j).second++;
            }
        }
    }

    //otherwise, ignore the random number because it does not land within the
    widths of the nearest bin
} //end loop

return histo;
}

```

/** THIRD PROBLEM

* desc: Given a list of students who have not found a group, place them in a group or create a new one if there are no groups available

* All students in the same group must be enrolled in the same section

* Groups can have no more than 3 members

* note: Students have Name, Section

* in: vector(list) of pairs(student)

* out: pass by pointer vector(list) of vector(groups) of pairs(student)

*/

void groupAssign(std::vector<std::vector<std::pair<std::string, int>>> &currList,

```

std::vector<std::pair<std::string, int>> newList){
    //init
    int i;
    int j;
    bool assigned;

    //for each student in newList
    for(i = 0; i < newList.size(); i++){
        //at each group in currList or until a group is found
        //initialize assigned status
        assigned = false;
        for(j = 0; j < currList.size() && !assigned; j++){
            //check that the group is not full
            //and check if a person from the group has a matching section
            if(currList.at(j).size() < 3
            && currList.at(j).at(0).second == newList.at(i).second){
                //add the element to the group in currList
                currList.at(j).push_back(newList.at(i));
                //student is now assigned, leave loop early
                assigned = true;
            }
            //if check failed: this group is either full or wrong section. move onto
next group
        }

        //if assigned is still false by the time it reaches the end of group loop,
then a new group must be created
        if(!assigned){
            //create group
            std::vector<std::pair<std::string, int>> newGroup(0);
            //add the student to the new group
            newGroup.push_back(newList.at(i));
            //add new group to the list of groups
            currList.push_back(newGroup);
        }
    }
}

```

/** UNIT TESTS */

/* Normal Distribution Test */

void normDistTest(double mean, double stdDev, int numBins, int numSamples, double binWidth){

std::vector<std::pair<double, int>> fin = normalDistribution(mean, stdDev, numBins, numSamples, binWidth);

```

    for(int i = 0; i < fin.size(); i++){
        std::cout << fin.at(i).first << ": ";
        for(int j = 0; j < fin.at(i).second; j++){
            std::cout << " ";

```

```

    }
    std::cout << std::endl;
}
}

```

```

/* Uniform Distribution Test */
void unifDistTest(double a, double b, int numBins, int numSamples, double binWidth){
    std::vector<std::pair<double, int>> fin = uniformDistribution(a, b, numBins,
numSamples, binWidth);
    for(int i = 0; i < fin.size(); i++){
        std::cout << fin.at(i).first << ": ";
        for(int j = 0; j < fin.at(i).second; j++){
            std::cout << "*";
        }
        std::cout << std::endl;
    }
}
}

```

```

/* Group Assignment Test */
void groupAssignTest(std::vector<std::vector<std::pair<std::string, int>>> current,
std::vector<std::pair<std::string, int>> newL){
    groupAssign(current, newL);
    for(int i = 0; i < current.size(); i++){
        std::cout << "Group " << i+1 << ": ";
        for(int j = 0; j < current.at(i).size(); j++){
            std::cout << current.at(i).at(j).first << " | ";
        }
        std::cout << "Section " << current.at(i).at(0).second << std::endl;
    }
}
}

```

```

/** MAIN METHOD */
int main(){

    normDistTest(5, 5, 9, 30, -1);
    std::cout << std::endl;

    unifDistTest(1, 10, 10, 30, -1);
    std::cout << std::endl;

    std::vector<std::vector<std::pair<std::string, int>>> myList;
    std::vector<std::pair<std::string, int>> myGroup;
    std::pair<std::string, int> student;
    //group 1
    student.first = "Phong";
    student.second = 3;
    myGroup.push_back(student);
}

```

```
student.first = "Rathang";
student.second = 3;
myGroup.push_back(student);
student.first = "Michael";
student.second = 3;
myGroup.push_back(student);
myList.push_back(myGroup);
myGroup.clear();
//group 2
student.first = "abc";
student.second = 2;
myGroup.push_back(student);
student.first = "def";
student.second = 2;
myGroup.push_back(student);
myList.push_back(myGroup);
myGroup.clear();
//new group
student.first = "zyx";
student.second = 3;
myGroup.push_back(student);
student.first = "wvu";
student.second = 2;
myGroup.push_back(student);

groupAssignTest(myList, myGroup);
return 0;
```

```
}
```